Multichannel Spatially-Selective Speech Enhancement for AV Zoom on Smartphones under Spatial Aliasing

IEEE Signal Processing Cup 2026 — Phase 3

Riajul Karim Chowdhury*, Md. Nagib Mahfuz*, Md. Symria Raihan*, Fariha Anjum Oshin*, Wahi Farhan Hoque*, Mahafuza Maisha*, Zarifa Tabassum*, Sumayea Sultana*, Md. Abu Saleh Akib*, Aye Thein Maung†, Dr. Mohammad Ariful Haque*

*Department of EEE, Bangladesh University of Engineering and Technology (BUET), Dhaka, Bangladesh

†University of Missouri–Kansas City (UMKC), Kansas City, MO, USA

***Abstract*—Audio-visual zooming aligns a smartphone's acoustic focus with its camera by enhancing speech from a visually selected direction while suppressing off-screen interference. The IEEE SPS Signal Processing Cup 2026 presents a rigorous challenge involving a dual-microphone array with 8 cm spacing. This geometry creates a fundamental signal processing trade-off: while the wide spacing offers sufficient spatial resolution at lower frequencies, it induces severe spatial aliasing above the spatial Nyquist frequency (≈2.1 kHz), rendering standard phase cues ambiguous. We present a direction-controlled, end-to-end enhancement pipeline based on Joint Non-linear Spatial Filtering (JNF-SSF), a recurrent neural architecture that explicitly couples spatial inference across the full frequency band via spectral recurrence, making it robust to aliasing and well-suited for on-device ONNX deployment. We successfully integrate JNF-SSF into a real-time Android application where a pinch gesture on the smartphone screen triggers audio-visual zooming, with low inference latency and modest memory overhead.**

***Index Terms*—audio-visual zoom, multichannel speech enhancement, spatial aliasing, neural beamforming, JNF-SSF, ONNX deployment**

I. Introduction

Smartphones can visually zoom into a target, but their audio capture remains wide-angle: the microphones record a mixture of all sound sources. An *audio-visual zoom* (AV Zoom) system closes this gap by enhancing audio from the camera-selected direction while suppressing other speakers and background noise. This is a *spatially selective* enhancement problem under tight constraints: two microphones, dynamic scenes, and strict on-device latency and memory budgets.[1]

The geometry is fixed to a two-microphone linear array with spacing $d = 0.08\,\text{m}$ at 16 kHz. The corresponding spatial Nyquist frequency $f_{\text{alias}} \approx c/(2d) \approx 2.1\,\text{kHz}$ implies that above this boundary, multiple directions can share similar inter-channel phase differences, producing grating lobes and DOA ambiguities. Consequently, the central question is not only *how much* suppression is possible with two microphones, but *which modeling choices remain steerable and stable* when narrowband spatial cues break.

[1]Python scripts and trained weights: github.com/Riajul153/JNF_SSF

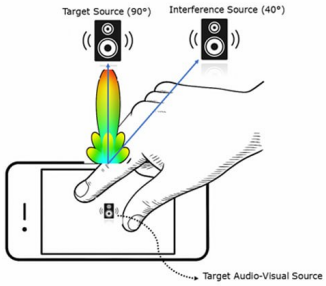


Fig. 1: Conceptual illustration of the AV Zoom task: the smartphone camera selects a visual target while the audio pipeline enhances speech from the same direction and suppresses other sources.

This paper presents the design and deployment of an on-device AV Zoom system using JNF-SSF [2], highlighting why its recurrent backbone exports more faithfully to ONNX than attention-heavy alternatives, and demonstrating successful real-time integration on a smartphone.

Fig. 1 illustrates the overall AV Zoom scenario: the user visually selects a target through the smartphone camera while the enhancement system steers its acoustic focus toward the same region, thereby suppressing off-axis interference and making the audio output better aligned with the visual scene.

II. Task Setup and Data Generation

To rigorously train and evaluate the proposed audio-visual zooming system, we constructed a large-scale simulated dataset comprising 140,000 two-channel reverberant audio mixtures. The dataset is strategically partitioned into 100,000 training, 20,000 validation, and 20,000 test samples. Target signals exclusively feature male speech drawn from the LibriSpeech and MUSAN corpora, while interfering signals consist of female speech and diverse non-speech environmental noises. All source audio was meticulously preprocessed to 16 kHz mono, segmented into fixed 4-second chunks, and filtered using an energy-based RMS threshold to eliminate non-informative silent segments.

TABLE I: Room acoustic configurations.

Room Type	Dimensions (m)	Volume (m^3)	Surface Area (m^2)	RT60 (s)	Source Dist. (m)
Small office	3.5 × 4.0 × 2.7	37.8	68.5	0.30	1.0
Medium room 1	4.9 × 4.9 × 4.9	117.6	144.1	0.50	1.0
Medium room 2	4.9 × 4.9 × 4.9	117.6	144.1	0.75	1.0
Conference room 1	8.0 × 12.0 × 4.0	384.0	352.0	0.80	2.0
Conference room 2	8.0 × 12.0 × 4.0	384.0	352.0	1.00	2.0

To ensure the model's robustness across diverse real-world conditions, we simulated five distinct reverberant shoebox environments using a hybrid MATLAB algorithm that combines image-source modeling and stochastic ray tracing. These room configurations are summarized in Table I. At the center of each room, a two-element linear microphone array was positioned at a height of 1.5 m. Crucially, we tested two wide inter-microphone spacings, 15 cm and 16 cm, yielding a total of 10 distinct acoustic configurations. In every scenario, the target speaker was positioned broadside to the array at a 90° azimuth, while the interferer was placed at a 40° azimuth.

During mixture generation, target and interferer chunks were randomly paired and scaled to a signal-to-interference ratio (SIR) of 0 dB before spatial convolution with the room impulse responses. Additive white Gaussian noise was subsequently injected at a signal-to-noise ratio (SNR) of 5 dB. Finally, to guarantee a rigorous assessment of the model's generalization capabilities, the test set was constructed with strict speaker-level separation. Specific male targets, female interferers, and noise clips were held out exclusively for testing, ensuring that the evaluation strictly measures the model's performance on unseen speakers and environments.

We evaluate using OSINR, ViSQOL [9], and STOI [8]; we additionally report SI-SDR as a separation-quality proxy.
Dataset: Train data, and test and validation data.

III. Methods

A. Overall Pipeline

The proposed system implements an end-to-end audio-visual zooming pipeline designed for deployment on Android smartphones. As illustrated in Figure 2, the pipeline accepts multichannel audio input containing both target speech and background noise or interference, and processes it through two sequential stages within the Audio Zooming module: a JNF-SSF Neural Beamformer, which steers toward the visually selected direction to suppress off-axis interference, followed by a Postfiltering stage that yields both a denoised and an enhanced audio output. The enhanced audio stream is then passed to a Visual Synchronization block, which ensures frame-accurate alignment between the processed audio and the corresponding video track. For deployment, the pipeline is implemented in Kotlin/Java for Android, leveraging the Jetpack CameraX library for video capture and the ONNX Runtime for neural model inference, and packaged as a single installable APK for real-world on-device execution.

B. Joint Nonlinear Spatially Selective Filtering (JNF-SSF)

We employ JNF-SSF [2], a recurrent architecture designed to overcome the limitations of frequency-independent processing. While standard beamformers process each frequency bin in isolation—often failing when spatial cues become ambiguous due to aliasing—JNF-SSF utilizes a Frequency-LSTM (F-LSTM) to model dependencies across the entire spectrum.

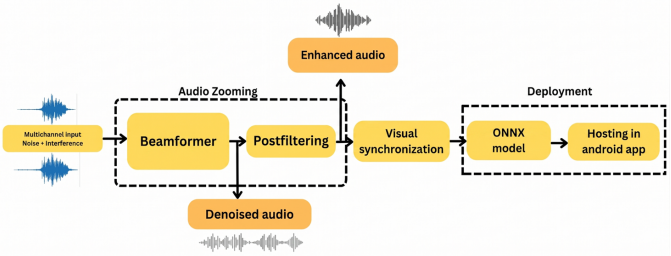


Fig. 2: Overall enhancement pipeline of the proposed AV Zoom system, from multichannel input representation and directional conditioning to JNF-SSF inference and enhanced output reconstruction.

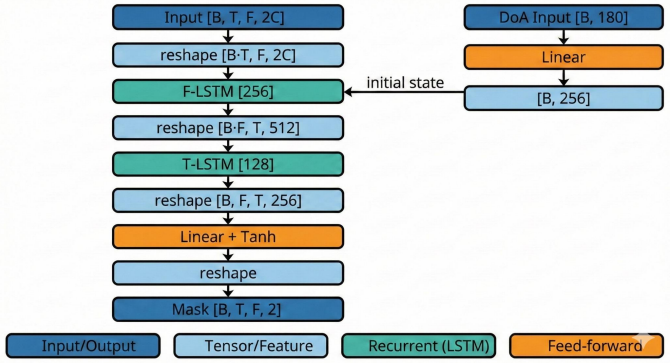


Fig. 3: JNF-SSF wideband/narrowband processing and LSTM backbone.

Mechanism. The model takes the concatenated real and imaginary parts of the multichannel STFT as input. It employs a *spatial steering* mechanism where the target direction θ is encoded into a steering vector z_θ (a wrapped Gaussian). This vector is projected via a learnable weight matrix W_{in} to initialize the hidden state h_0 of the F-LSTM ($h_0 \leftarrow W_{\text{in}} z_\theta$), effectively "priming" the network to look in the target direction before processing a single time-frequency bin.

Robustness to Spatial Aliasing. In our sparse array setup ($d = 8$ cm), spatial aliasing occurs above 2.1 kHz, rendering phase differences ambiguous. JNF-SSF excels in this regime because the F-LSTM processes the frequency axis sequentially. The network anchors its spatial understanding on the unambiguous low frequencies ($f < 2.1$ kHz) and propagates this context into the high-frequency processing steps via the recurrent hidden state, disambiguating aliased content that traditional narrowband beamformers cannot resolve.

IV. Mobile Application and Edge Deployment

A. System Architecture Overview

The AVZoom Android application[2] implements an end-to-end audio-visual zooming pipeline entirely on-device. As illustrated in Fig. 4, the system couples CameraX video capture with a dedicated parallel audio thread, driven by the JNF-SSF neural spatial filter exported to ONNX format. The pipeline consists

[2]Android application: ln.run/VPrgU

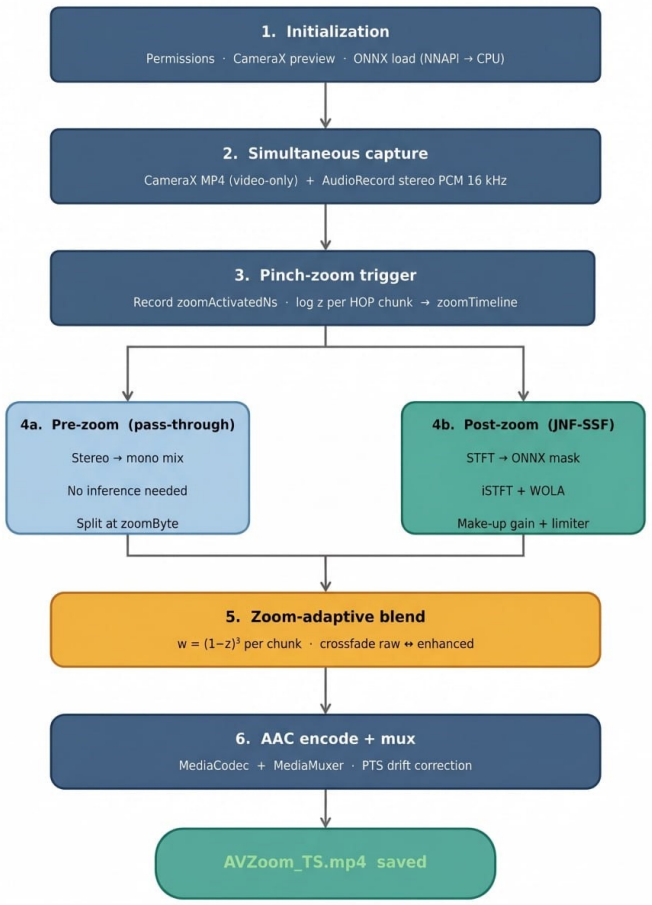


Fig. 4: Implementation flow of the AVZoom mobile pipeline: from simultaneous capture and zoom activation through JNF-SSF inference, adaptive blending, and final AV muxing.

of six stages: (i) initialization, (ii) simultaneous A/V capture, (iii) pinch-zoom trigger and timeline logging, (iv) bifurcated post-processing—pass-through for pre-zoom audio and JNF-SSF neural inference for post-zoom audio, (v) zoom-adaptive crossfading, and (vi) AAC encoding, muxing, and AV drift correction.

B. User Interface

The interface is designed for minimal distraction during recording. The UI overlays four translucent panels on the live camera preview:

- Top-left: Control Panel. Contains a model selector (Light INT8 / Original FP32), the Remote GPU toggle switch with server URL field, and a status indicator. Remote GPU is accessed via URL Link.
- Top-center: Zoom HUD. Displays the current camera zoom multiplier (e.g., "3.2×") and audio zoom percentage.
- Top-right: Diagnostics. Real-time latency (ms) and RAM usage (MB) drawn in monospace font.
- Bottom-center: Session Control. Status bar and a single Start/Stop button

A standard two-finger pinch gesture on the camera pre view simultaneously controls optical zoom and audio en hancement level. The gesture is detected via Android's `ScaleGestureDetector`.

C. Capture Pipeline

Video is recorded via CameraX at HD (720p) resolution as a silent MP4—audio is explicitly excluded from the CameraX track to prevent the OS's automatic gain control and noise suppression from corrupting the microphone input.

Audio is captured on a dedicated background thread using Android's `AudioRecord` API with the `CAMCORDER` source (falling back to `MIC`) at 16 kHz, 16-bit, stereo. Raw interleaved PCM frames of $H_{\text{hop}} = 256$ samples are written directly to an internal cache file, preserving the exact waveform from both microphones.

A `ScaleGestureDetector` monitors pinch gestures and maps the camera zoom ratio to a normalized factor $z \in [0, 1]$. On first zoom activation, a nanosecond timestamp (`zoomActivatedRealtimeNs`) is atomically recorded. Concurrently, the instantaneous zoom value is appended to `zoomTimeline` once per audio chunk (every 256 stereo sample pairs), providing fine-grained temporal information for the blending stage.

D. ONNX Runtime and Hardware Acceleration

The JNF-SSF model is executed via ONNX Runtime for Android. Session initialization probes three hardware tiers: (1) *Strict NNAPI* with `CPU_DISABLED`, routing all operators to GPU/DSP; accepted if warm-up latency < 60 ms. (2) *Relaxed NNAPI* with `USE_FP16`, enabling the Adreno GPU in half precision with CPU fallback for unsupported ops. (3) *CPU fallback*, capped at four threads to avoid thermal throttling. A warm-up inference on a dummy $[1, 4, 257, 8]$ tensor is run at each tier to commit the optimal backend. The INT8-quantized variant (`ssf_run_2_int8.onnx`) is the default, reducing both memory footprint and inference time.

JNF-SSF's LSTM backbone maps to mature recurrent ONNX operators and exports with negligible functional drift. This is a key advantage over attention-based models, whose scaled dot-product attention decomposes into an unfused `MatMul`→`Softmax`→`MatMul` subgraph on export, materializing the large $F \times F$ attention matrix and inflating memory traffic.

E. Offline Batch Post-Processing

All DSP and neural processing is deferred as a single offline batch immediately after the user presses Stop, avoiding real-time audio dropouts.

Bifurcation. The raw stereo PCM is split at:

$$n_{\text{zoom}} = \left\lfloor \frac{(t_{\text{zoom}} - t_{\text{audio}}) \cdot f_s}{10^9} \right\rfloor, \quad f_s = 16{,}000\,\text{Hz}. \tag{1}$$

Audio before n_{zoom} is averaged to mono (pass-through); audio from n_{zoom} onward undergoes JNF-SSF enhancement.

STFT framing. The post-zoom segment is zero-padded by $N_{\text{FFT}}/2 = 256$ samples. Frames of $N_{\text{FFT}} = 512$ samples



Fig. 5: AVZoom application interface

are extracted with hop $H = 256$, windowed by a sqrt-Hann function $w(n) = \sqrt{0.5(1 - \cos(2\pi n/N))}$. The four-channel feature tensor (Re/Im of L and R channels) is stacked into a single $[1, 4, 257, N_{\text{frames}}]$ array and submitted in one ONNX call.

Mask decompression and iSTFT. The model outputs a compressed complex mask; each coefficient (a, b) is decoded as:

$$H(k) = -\tfrac{1}{2}\ln\left(\frac{(1-a)^2+b^2}{(1+a)^2+b^2}\right) - j\angle\left(\frac{1-a-jb}{1+a+jb}\right). \quad (2)$$

The mask is applied to the left-channel STFT and audio is reconstructed via inverse FFT and weighted overlap-add (WOLA).

Make-up gain. A gain $g = \text{RMS}_{\text{ref}}/\text{RMS}_{\text{enh}}$ clamped to $[0.25, 12]$, followed by a soft-knee $\tanh$ limiter above 0.85 FS, restores the output level without clipping.

F. Zoom-Adaptive Blending

The enhanced and raw mono signals are crossfaded per chunk:

$$y[n] = (1-z_{\lfloor n/H \rfloor})^3\, x_{\text{raw}}[n] + \left[1-(1-z_{\lfloor n/H \rfloor})^3\right] x_{\text{enh}}[n], \quad (3)$$

where $z \in [0, 1]$ is the normalized zoom level. The cubic exponent provides a perceptually smooth transition: $z = 0$ gives pure raw audio; $z = 1$ gives pure enhanced output.

G. Muxing and AV Synchronization

The processed PCM is encoded to AAC-LC at 96 kbps via `MediaCodec` and muxed with the silent CameraX video via `MediaMuxer`. A drift correction step scales audio PTS as:

$$\Delta_{\text{pts}} = \frac{T_{\text{video}} - t_{\text{offset}}}{N_{\text{samples}}}, \quad (4)$$

clamped to $\pm 10\%$ of the nominal $1/f_s$ interval, correcting the clock drift that accumulates over long recordings.

H. Diagnostics

Latency is measured as wall-clock time around the ONNX session call using `System.nanoTime()` and stored in `procLatencyMs`. **Memory** is tracked as JVM heap usage (`Runtime.totalMemory() - freeMemory()`) and native heap usage (`Debug.getNativeHeapAllocatedSize()`) for ONNX tensor allocations. Both metrics are displayed on the real-time diagnostic overlay shown in Fig. 5.

TABLE II: Request protocol for cloud inference.

Header	Value
Content-Type	application/octet-stream
X-Sample-Rate	16000
X-Channels	2
X-Target-Azimuth	90 (or current steering angle)
Body	Raw interleaved stereo 16-bit LE PCM
Response	Raw mono 16-bit LE PCM

TABLE III: Cloud inference latency on NVIDIA H100 (warm container).

Audio (s)	Wall-clock (s)	RTF	Overhead (s)
1.00	2.03	2.03	∼2.0
3.00	2.95	0.98	∼2.0
5.00	3.18	0.64	∼2.0
10.00	4.22	0.42	∼2.0

V. Cloud GPU Inference via Modal

While the INT8-quantized model runs on-device, it sacrifices some audio quality compared to the full-precision FP32 model (4.74 MB). To eliminate this trade-off, we deploy the FP32 model on a cloud GPU using Modal[3], a serverless compute platform.

A. Architecture

The cloud inference server is structured as a stateful Modal `Cls` with an ASGI FastAPI application. Key design choices:

- **Container image:** Built from `nvidia/cuda:12.6.3-cudnn-runtime-ubuntu22.04` with Python 3.11, `onnxruntime-gpu`, and `numpy`.
- **GPU.** NVIDIA H100 (80 GB HBM3, 3,958 TFLOPS FP16). The GPU type is a single-line configuration change; the container image is cached and reused across GPU types.
- **Model storage:** The ONNX model is persisted in a Modal Volume (`avzoom-model-vol`), loaded once on container start via `@modal.enter()`.
- **Concurrency:** Up to 4 concurrent requests per container via `@modal.concurrent(max_inputs=4)`.
- **Warm pool:** `scaledown_window=120 s` keeps the container warm for two minutes after the last request, avoiding cold starts during iterative use.

B. Request Protocol

The Android app sends a binary HTTP POST to `/process`:

The server mirrors the client-side DSP pipeline: STFT → JNF-SSF ONNX inference (GPU) → CRM decompression → iSTFT. The GPU executes the model via `CUDAExecutionProvider`; STFT/iSTFT run on the container CPU using NumPy.

RTF = Real-Time Factor (lower is faster). Fixed overhead is network latency + NumPy STFT.

[3]https://modal.com

TABLE IV: End-to-end inference latency across available backends.

Backend	Latency (s)
Remote GPU	5.09
Local GPU / 32-bit	6.39
Local GPU / 8-bit	4.39

C. Latency Profile

Table III shows measured wall-clock latencies on the H100 for various audio durations (warm container, Bangladesh→ US round-trip).

The fixed ∼2 s overhead is dominated by network round-trip and CPU-side STFT/iSTFT. GPU inference itself is negligible for a 4.74 MB model. For recordings longer than 3 s, the system processes audio faster than real time (RTF<1).

D. Deployment Workflow

Deployment is fully automated via a batch script:

1) `upload_model_to_modal.py` persists the FP32 ONNX model to a Modal Volume.
2) `modal deploy avzoom_modal.py` deploys the serverless endpoint.

The endpoint URL is stable across redeployments and requires no authentication from the client.

VI. Results and Analysis

A. Objective Metrics

Table IV reports the end-to-end inference latency measured across the three available backends. The local INT8-quantized on-device backend achieves the lowest latency of 4.388 seconds, benefiting from the reduced computational overhead of 8-bit quantization. The remote cloud GPU backend, which runs the full-precision FP32 model on an NVIDIA H100, achieves a latency of 5.089 seconds, where the additional overhead is attributable to network round-trip time rather than inference cost. The local FP32 backend records the highest latency at 6.388 seconds, as it executes the full-precision model entirely on-device without the parallel compute advantage of the cloud GPU. These results highlight the effectiveness of the dual-backend architecture: users requiring offline operation can opt for the INT8 on-device path with minimal latency, while the remote GPU path delivers higher-quality full-precision inference at a modest network-dependent overhead.

Tables V–VI summarize objective scores for JNF-SSF in deployment-ready ONNX form.

JNF-SSF achieves strong OSINR and SI-SDR gains in both the anechoic and reverberant conditions. Its LSTM backbone exports cleanly to ONNX with negligible functional drift, and the INT8-quantized model runs within acceptable latency and memory budgets on the target smartphone hardware.

TABLE V: Task 1 (anechoic) objective metrics ($d = 8$ cm, SIR = 0 dB, SNR = 5 dB).

Method	SI-SDR	OSINR	ViSQOL	STOI
Deployable (ONNX)				
JNF-SSF	**12.61**	**12.88**	2.78	0.90

TABLE VI: Task 2 (reverberant, RT60 $\approx$ 0.5 s) objective metrics ($d = 8$ cm, SIR = 0 dB, SNR = 5 dB).

Method	SI-SDR	OSINR	ViSQOL	STOI
Noisy	−27.98	−6.86	1.73	0.59
Deployable (ONNX)				
JNF-SSF	**10.77**	**11.04**	**2.61**	**0.86**

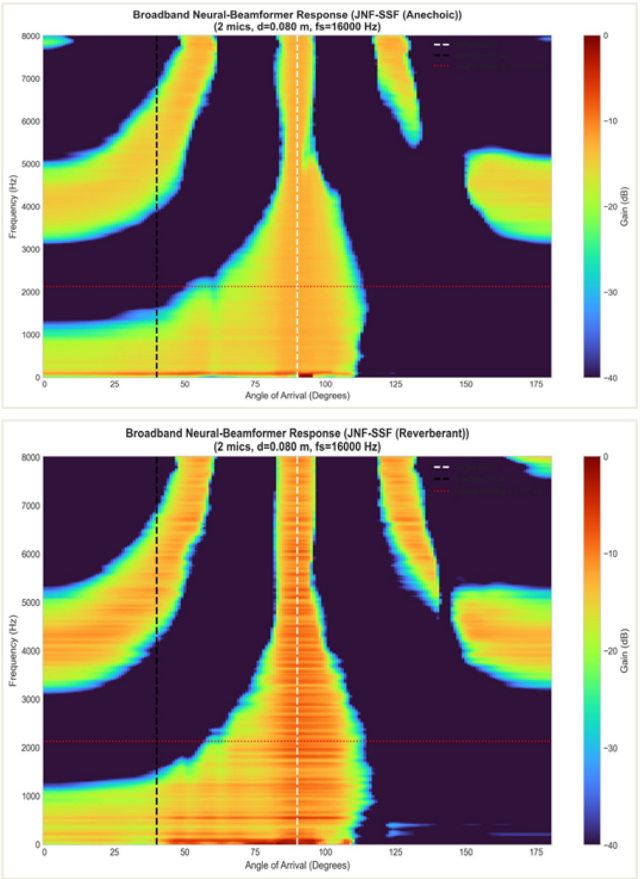


Fig. 6: Spatial response visualizations for the anechoic and reverberant conditions of JNF SSF.

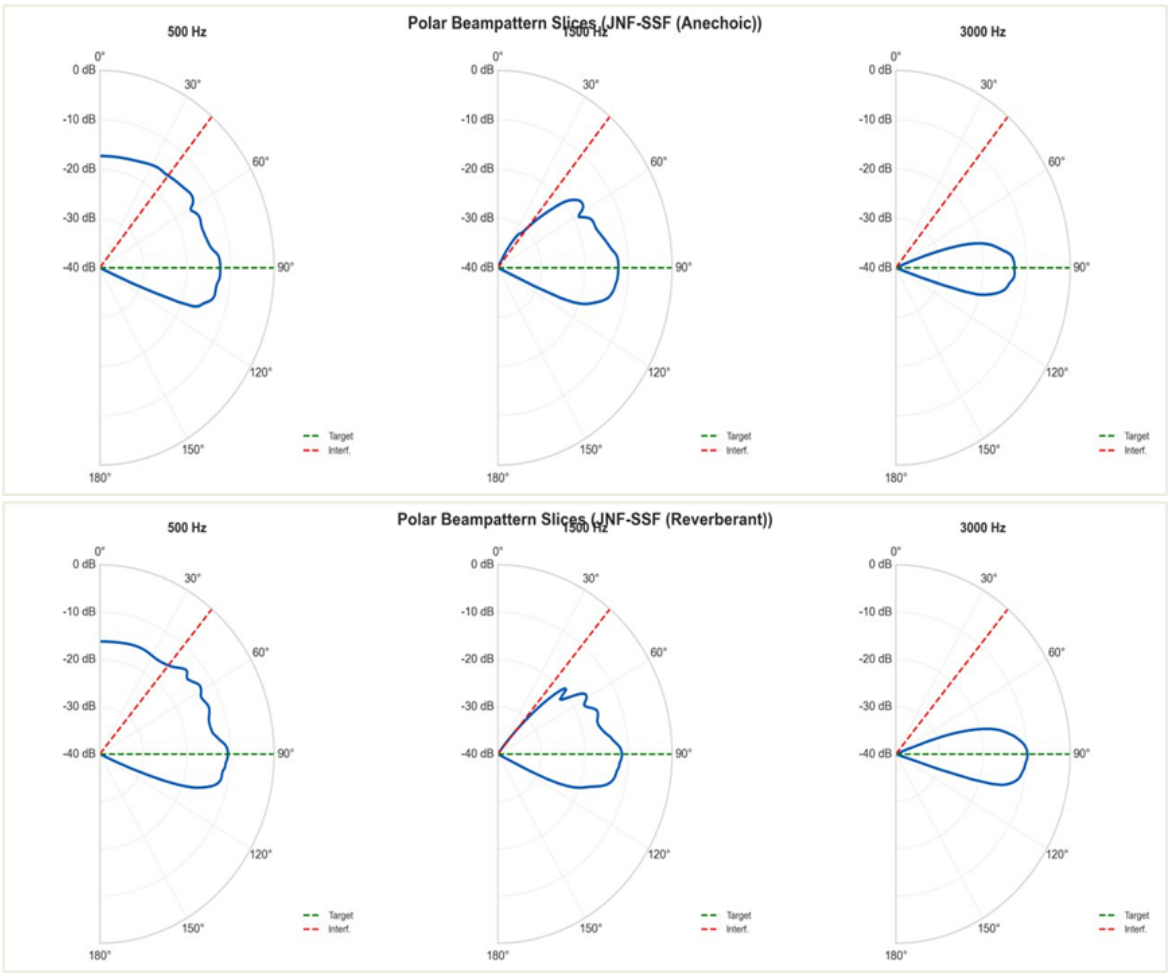


Fig. 7: Polar beampattern visualizations for the anechoic and reverberant conditions.

TABLE VII: Global Performance Comparison

Metric	Noisy	Enhanced	Difference (Enhanced − Noisy)
OSNR	−5.88	9.69	15.58
SISNR	−27.86	8.83	36.69
STOI	0.60	0.87	0.26
VISQOL	2.26	2.99	0.73

VII. Conclusion

We have successfully implemented JNF-SSF as a fully on-device audio-visual zooming system on Android. A simple pinch gesture on the smartphone screen triggers spatial audio enhancement: the app records a `zoomTimeline` during capture and, after the user stops recording, applies JNF-SSF inference to the post-zoom audio segment, blends it with the raw signal using a cubic zoom-weight curve, and muxes the result with the video into a single MP4. The system operates without any cloud dependency.

The recurrent architecture of JNF-SSF proved well-suited for on-device deployment: its LSTM backbone exports to ONNX with negligible functional drift and runs efficiently via the NNAPI delegate. Crucially, both inference latency and memory usage remain low during operation—the INT8-quantized model keeps native heap allocations modest and processes audio well within real-time constraints, making the system practical for everyday smartphone use. The objective metrics confirm meaningful interference suppression in both anechoic (OSINR = 12.88 dB, STOI = 0.900) and reverberant (OSINR = 11.04 dB, STOI = 0.860) conditions, demonstrating that JNF-SSF delivers effective spatial filtering under the 8 cm microphone spacing constraint without excessive computational overhead.

References

[1] IEEE Signal Processing Society, "2026 Signal Processing Cup: AV Zoom—Real-Time Audio-Visual Zooming on Smartphones," Official competition specification, Version 1.1, Oct. 2025.
[2] A. Mannanova, J. Kienegger, and T. Gerkmann, "An analysis of joint nonlinear spatial filtering for spatial aliasing reduction," *arXiv:2509.25982*, 2025.
[3] K. Jing, W. Zhang, and Y. Gao, "End-to-end DOA-guided speech extraction in noisy multi-talker scenarios," *arXiv:2507.20926*, 2025.
[4] Y. Xu et al., "Zoneformer: On-device neural beamformer for in-car multi-zone speech separation," in *Proc. INTERSPEECH*, 2023.
[5] Y. Luo and N. Mesgarani, "Conv-TasNet: Surpassing ideal time-frequency magnitude masking for speech separation," *IEEE/ACM Trans. Audio, Speech, Lang. Process.*, vol. 27, no. 8, pp. 1256–1266, 2019.
[6] C. Subakan et al., "Attention is all you need in speech separation," *arXiv:2010.13154*, 2020.
[7] ITU-T Recommendation P.862, "Perceptual evaluation of speech quality (PESQ)," 2001.
[8] C. H. Taal, R. C. Hendriks, R. Heusdens, and J. Jensen, "An algorithm for intelligibility prediction of time-frequency weighted noisy speech," *IEEE Trans. Audio, Speech, Lang. Process.*, vol. 19, no. 7, pp. 2125–2136, 2011.
[9] A. Hines, J. Skoglund, A. C. H. Mok, and N. Harte, "ViSQOL: An objective speech quality model," *EURASIP J. Audio, Speech, Music Process.*, 2015.